FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

FH Aachen

Bachelor's Thesis

in partial fulfillment of the requirements for the degree of

B. Sc. in Computer Science

AI Agents in Software Development: Best Practices for Structured Human-Agent Collaboration

Fakih Lanjri Houdaifa

Matriculation Number 3574047 Aachen,

First Examiner:

Second Examiner:

Declaration

I hereby declare that I have written this thesis independently and have not used any sources other than those indicated in the bibliography. Passages that have been taken verbatim or in substance from published or as yet unpublished sources are identified as such. The drawings or illustrations in this thesis were created by myself or are provided with an appropriate reference to their source. This thesis has not been submitted in the same or similar form to any other examination authority.

Aachen,

Contents

1 Introduction **5**
1.1 Background 5
1.2 Motivation 5
1.3 Problem Statement 6
1.4 Objectives of the Thesis 7

2 Foundations **9**
2.1 Large Language Models in Software Engineering 9
2.2 AI Coding Agents and Agentic Workflows 12
2.3 Multi-Agent Systems and Role Specialization 14
2.4 Human-in-the-Loop and Human Oversight in AI-Assisted Development 16
2.5 Prompt Engineering 18
2.6 Software Architecture in the Context of AI-Generated Code 20
2.7 Software Testing and Test Validity 22
2.8 OpenAPI Specifications and Documentation Drift 24

3 Methodology **27**
3.1 Structure of the Practical Project (Two Sub-Projects) 27
3.2 Tools and Models Used 28
3.3 Evaluation Criteria 28

4 Part 1: Unstructured Development **29**
4.1 Initial Situation and Objectives 29
4.2 Prompting Without a Structural Approach 29
4.3 System Prompt Context 29
4.4 Development of the First Feature (Login/Registration) 29
4.5 Test Suites and Test Case Generation 29
4.6 Architectural Analysis of the Generated Software 29
4.7 Observations: Prompt Precision, Human Oversight, Architectural Limitations . . 29
4.8 Critical Problems and the Test Failure Analysis Principle" 29

5 Part 2: Structured Approach (AntiGravity) **30**
5.1 Motivation for a Structured Approach 30

5.2 AI Development Charter and Architectural Rules 30
5.3 Parallel Agent Architecture (Architect / Implementer / Post-Implementation Review Agent) 30
5.4 Design Reviews Before Coding (implementation_plan.md) 30
5.5 Testing Strategy: Force Requirement Validation & Three-Tier Testing 30
5.6 Project Knowledge Files 30
5.7 OpenAPI Specification Automation and Avoiding Documentation Drift 30
5.8 Identification of Architectural Problems and Weaknesses 30

6 Comparison of Both Approaches 31
6.1 Comparison: Part 1 vs. Part 2 31
6.2 Feature Diversity 31

7 Evaluation 32
7.1 Evaluation Criteria and Methodology 32
7.2 Quality of Generated Code 32
7.3 Quality and Validity of Generated Tests 32
7.4 Architectural Conformance and Documentation Consistency 32
7.5 Efficiency and Effort (Human vs. Agent) 32
7.6 Summary Assessment: Part 1 vs. Part 2 32

8 Best Practices for Human-Agent Collaboration 33
8.1 Importance of Prompt Precision and Context Design 33
8.2 Role of Human Oversight (Human-in-the-Loop) 33
8.3 Architectural Guardrails for AI Agents 33
8.4 Recommendations for Multi-Agent Workflows 33
8.5 Avoiding Documentation Drift 33

9 Discussion 34
9.1 Opportunities and Limitations of AI Agents in Software Development 34
9.2 Implications for Practice 34

10 Conclusion and Outlook 35

Bibliography 36

1 Introduction

1.1 Background

Large Language Models have moved a long way from being simple autocomplete engines for code. Today, tools built on top of them act as genuine coding agents: they can read a set of requirements, plan an implementation, write entire features, generate their own tests, and reason, at least to some degree, about the architecture of the software they are building. Modern coding agents can be integrated directly into conventional development environments, allowing developers to hand off substantial parts of the implementation work to an AI agent while still reviewing and supervising what it produces.

More recently, agent-first development environments have emerged that extend this concept further. Rather than relying on a single agent working through a linear sequence of tasks, these environments can support the orchestration of several specialized agents, each responsible for different parts of the development workflow, from planning and architectural design to implementation and verification.

This shift changes the nature of software development in a subtle but important way. In traditional development, the person writing the code is also the one reasoning about it. When an AI agent takes over the implementation, a layer is inserted between the idea and the code: the agent interprets instructions, makes its own architectural choices, and produces artifacts that a human then has to review, correct, or approve. How good the resulting software turns out to be depends not only on how capable the underlying model is, but just as much on how the collaboration between human and agent is set up in the first place — how precise the prompts are, whether any architectural boundaries are defined at all, and what kind of review process, if any, the agent's work has to pass.

1.2 Motivation

This thesis grew out of a practical project in which software was built from the ground up using AI coding agents, across two different development environments that each embody a different philosophy of working with an agent.

The first part of the project was carried out with Claude Code inside Visual Studio Code. Development here followed a deliberately loose approach: the agent was asked to implement features from scratch with very little structural guidance, beyond a general description of the software idea, the functional and non-functional requirements, and the core modules of the web application. The goal was to observe how an AI agent behaves when given substantial implementation freedom within a single-agent, conversational workflow — and where exactly that freedom starts to cause problems, whether in the form of inconsistency, architectural drift, or small but consequential implementation mistakes.

What came out of that first part shaped the second one directly. For the second part, development moved to Google Antigravity, an agent-first development environment organized around orchestrating multiple agents rather than interacting with a single one. Instead of assuming that better software simply requires a more capable model, this second phase focused on something different: the architectural rules, review steps, and development discipline imposed on the agents themselves. A more structured methodology was introduced for this purpose, built around an AI Development Charter, a three-agent setup (an Architect, an Implementer, and a Post-Implementation Architecture Review Agent), mandatory design reviews before any code was written, and a formal testing strategy — all of it designed to make deliberate use of the environment's support for parallel, role-specialized agents working together, rather than one agent working through a single, linear thread of tasks.

The motivation behind this thesis is therefore twofold. First, it aims to document and compare, in concrete terms, what changes when moving from an unstructured, single-agent approach to a structured, multi-agent approach in AI-assisted software development. Second, it aims to distill from that comparison a set of best practices for human-agent collaboration that hold up beyond the specific tools used here, and that can inform how developers approach this kind of work more generally.

1.3 Problem Statement

AI coding agents are being increasingly adopted in both industry and academic settings. However, systematic understanding of how the structure of human-agent collaboration — rather than solely the capability of the underlying model — influences the quality, architectural soundness, and reliability of resulting software remains limited. Existing research on AI-assisted software development has often focused on model capabilities, benchmarks, or individual code-generation tasks, while broader aspects of the development process —

including requirements interpretation, architectural planning, iterative implementation, testing, review, and documentation maintenance — require further investigation.

Several specific challenges motivate the work presented in this thesis:

1. **Limited structural comparison.** Existing work provides limited empirical comparison between unstructured, prompt-driven interaction with AI coding agents and more structured, architecture-first development approaches, particularly when both approaches are examined within the same software project.

2. **Unclear impact of architectural constraints.** It remains unclear to what extent architectural rules, mandatory design reviews, and the separation of responsibilities across specialized agents can improve the quality and consistency of AI-generated software compared with a single agent operating with fewer structural constraints.

3. **Gaps in testing and validation.** AI coding agents can generate tests that execute successfully without necessarily providing sufficient validation of the intended behavior. This raises the question of how testing and verification should be structured when substantial parts of both implementation and test generation are delegated to AI agents.

4. **Documentation drift.** As AI agents modify software iteratively, maintaining consistency between the implementation and related specifications, such as OpenAPI documents, becomes an important maintainability challenge.

5. **Limited consolidated guidance.** Despite the growing adoption of AI coding agents, empirically grounded guidance on how developers should structure human-agent collaboration across prompting, architecture, review, testing, and documentation remains fragmented.

1.4 Objectives of the Thesis

The aim of this thesis is to systematically document, analyze, and compare two contrasting approaches to AI-agent-assisted software development — one unstructured and one structured — based on a practical project, and to derive from this comparison concrete, actionable best practices for human-agent collaboration in software engineering.

More specifically, this thesis sets out to:

1. **Document the unstructured approach (Part 1).** Describe how the software was developed using a single-agent workflow with minimal architectural guidance, and analyze the resulting architectural limitations, the influence of prompt precision, and the extent to which human supervision was required.

2. **Document the structured approach (Part 2).** Describe the design and implementation of a structured, multi-agent development methodology, including the AI Development Charter, parallel role-specialized agents, mandatory design reviews, the three-tier testing strategy, and the automation of OpenAPI specifications to prevent documentation drift.

3. **Evaluate each approach.** Define evaluation criteria and assess each development approach with respect to code quality, test validity, architectural consistency, documentation consistency, and development efficiency.

4. **Compare the two approaches.** Conduct a direct cross-approach comparison to determine how the structured methodology affected the limitations observed during the unstructured development phase, including differences in feature implementation, code and test quality, architectural consistency, and documentation reliability.

5. **Derive best practices.** Bring the findings together into a set of best practices for human-agent collaboration, covering prompt precision, the role of human oversight, architectural guardrails, recommendations for multi-agent workflows, and strategies for preventing documentation drift.

Taken together, these objectives position the thesis as both an empirical case study of AI-agent-assisted software development and a practically oriented source of recommendations that developers and organizations can draw on when structuring their own collaboration with AI coding agents.

2 Foundations

2.1 Large Language Models in Software Engineering

Large Language Models (LLMs) are, at their core, autoregressive sequence generators: given a sequence of tokens as input, the model predicts a probability distribution over the next token and generates output by repeatedly sampling from this distribution. Nearly all current LLMs are built on the Transformer architecture, which underlies their ability to generate long, coherent sequences of text — including code.

When applied to source code, this generative mechanism is unchanged in principle: code is treated as a token sequence like any other. Just as a language model predicts the next word of a sentence from the words that came before it, a code-generating model predicts the next token of a program from the tokens that came before it — where a token may be a keyword, a variable name, a punctuation mark, or a fragment of one. Given the beginning of a function signature such as **`def calculate_total(`**, the model does not "know" what a function is in any formal sense; it predicts, token by token, that a parameter name is likely to follow, then a closing parenthesis, then a colon, then an indented block — because sequences with this shape occurred overwhelmingly often in the code it was trained on. The model's fluency with syntax, naming conventions, and common patterns is therefore a learned statistical regularity, not an explicit understanding of a programming language's grammar or semantics.

The practical capability of LLMs applied to code has developed considerably since general-purpose language models were first adapted to this domain. Early applications were largely limited to local code completion: a model would suggest the rest of a line, or perhaps the body of a short function, based only on the few lines immediately surrounding the cursor — closer to a smarter version of an IDE's autocomplete than to an independent programmer. A notable step beyond this was Codex, a GPT-based model that started as a general-purpose language model and was then further trained — fine-tuned — specifically on publicly available code from GitHub, which made it capable of writing Python code from natural-language docstrings alone. This meant the model could take nothing but a function signature and a description of the intended behavior, such as python

```python
def is_palindrome(s: str) -> bool:
    """Check if a string is a palindrome, ignoring
    case and non-alphanumeric characters."""
```

and generate a complete, working implementation beneath it — predicting, token by token, what code plausibly follows such a docstring, based on patterns learned from the enormous number of similar functions it had seen during training. On the HumanEval benchmark introduced alongside it, Codex solved 28.8% of hand-written programming problems from their descriptions alone when generating a single attempt. When the model was instead allowed to generate up to 100 candidate solutions per problem and only one needed to pass, its success rate rose to 70.2% — an early illustration that a single generated output is often unreliable on its own. This shift — from finishing a line to generating a working implementation from a description of intended behavior — is what makes it possible, in principle, to prompt a model with a feature description and receive an entire piece of software back, rather than a suggestion for what to type next. It is precisely this capability that current coding agents build upon.

These structural properties belong to the model itself, considered independently of any system built around it. A "call" to an LLM — a single request that sends in some text and receives generated text back — has no access to a filesystem, cannot execute code, and does not persist anything once it finishes responding, unless additional tooling is deliberately built around it to provide these capabilities. The limitations described below apply to the model in this bare form:

- **Fixed context window.** A model can only condition its output on a bounded amount of text supplied in a single call — its context window. Imagine asking a developer to fix a bug in one file of a large application, but showing them only that one file and none of the others it depends on: they might produce a change that looks correct in isolation while silently breaking an assumption made somewhere else in the codebase, simply because they never saw it. An LLM faces the same problem whenever a project's relevant files, conventions, or history exceed what fits in a single call. This limits the context available to the model when making decisions, can prevent it from reasoning across an entire unit of code at once, and means the model simply cannot process code that does not fit within the window unless the relevant information is explicitly and selectively supplied as input. This limitation is well documented specifically for coding LLMs: the difficulty of maintaining context over long sequences of code is considered a significant technical limitation of LLMs in code generation, since the functionality and correctness of code often depends on information that lies outside of what fits in a single window. A single LLM call therefore cannot be expected to hold an entire codebase, its full change history, or an extensive set of project-specific conventions in mind at once.

- **No persistent memory across calls.** Each generation is, by default, stateless: the model does not retain anything from a previous call unless that information is explicitly re-supplied as part of the current prompt. Concretely, if a developer asks a model to implement a login feature, and in a later, separate call asks it to write tests for "the login feature I just described," the model has no memory of the earlier conversation to draw on — from its perspective, "the feature I just described" refers to nothing at all, unless the original description is pasted back in. Any continuity across a multi-step development task must therefore be actively maintained by whatever system is orchestrating the calls, not by the model itself.

- **No inherent mechanism for verification.** An LLM generates output that is statistically plausible given its training and the supplied context, but it has no built-in means of executing that output, checking it against a specification, or confirming that it behaves as intended. This is a specific instance of a broader, well-studied phenomenon in LLM research known as hallucination — the generation of content that is fluent and syntactically well-formed but factually inaccurate or unsupported by the available evidence. Applied to code, this means a model can generate a function that reads as entirely plausible — correct indentation, sensible naming, no syntax errors — while still returning the wrong result for certain inputs, calling a method that doesn't exist, or silently failing to handle an edge case the requirement demanded. The model has no way of catching this itself, since doing so would require actually running the code and comparing its behavior against the specification — a step the model itself cannot perform. The same applies to generated tests: a test that is syntactically valid and executes without error is not automatically a test that meaningfully validates the intended behavior.

Taken together, these properties mark the boundary of what a bare LLM call can be expected to do: it provides a generative and reasoning capability over token sequences, including code, but has no native ability to interact with a filesystem, execute code, retain state across multiple turns, or verify the correctness of its own output. Any system that requires these capabilities — as agentic coding tools do — must supply them externally, layered on top of the model rather than as an inherent feature of it. This distinction motivates the notion of a coding agent, introduced in the following section.

2.2 AI Coding Agents and Agentic Workflows

A bare LLM call, as discussed above, cannot access a filesystem, execute code, retain memory across calls, or verify its own output. A coding agent is what results when a system is deliberately built around an LLM to supply exactly these missing capabilities. The LLM still provides the underlying reasoning and text-generation capability, but it is now embedded in a loop that lets it observe its environment, take concrete actions in it, and adjust its next step based on what happens — rather than producing a single block of text and stopping. A common way of describing this is in terms of three capabilities that, together, enable autonomous behavior: planning, memory, and tool use.

From single generation to an iterative loop. A widely used way of structuring this loop, introduced under the name ReAct, has the model cycle repeatedly through three types of output instead of producing a single final answer: a reasoning step describing its understanding of the problem and its plan, an action it takes to gather information or affect its environment, and an observation of the result of that action, which then feeds into the next reasoning step. Instead of a plain LLM call, which receives an input and produces one finished answer, an agent structured this way produces a reasoning step, then an action, then reads back an observation of what that action actually returned — and repeats this cycle, each round informed by the last, until it judges the task complete.

Applied to a coding task, this loop might look concretely as follows. Given the instruction **"add input validation to the registration form,"** an agent does not simply generate a finished file in one step:

1. Reasoning: the agent reasons that it first needs to see the existing form implementation before it can change it.
2. Action: it takes the action of reading that file.
3. Observation: it observes the file's actual content — the current fields, structure, and existing logic.
4. Reasoning: based on that observation, it reasons about where validation logic should be added.
5. Action: it takes the action of writing the change to the file.
6. Observation: it runs the project's test suite as a further action and observes whether the tests pass.
7. Reasoning: if a test fails, it reasons about why, and the cycle continues — reading further code, revising the change, re-running tests — until the observation confirms the task is complete.

Each individual step in this loop is still, underneath, a single LLM call — stateless, bounded by a context window, with no ability to verify anything on its own. What the agent adds is the surrounding structure that decides what to feed into the next call, based on what came back from the previous one, so that seven or more individual calls end up behaving like one continuous, self-correcting attempt at the task.

Tool use. The "actions" in this loop are only possible because the agent is given access to tools: concrete operations it can invoke, such as reading or writing a file, running a shell command, executing a test suite, or querying an external service. A representative example of teaching a model to use tools is Toolformer, a model trained to decide which external APIs to call, when to call them, what arguments to pass to them, and how to incorporate their results back into what it generates next. A coding agent applies the same underlying idea to a developer's toolbox instead of a calculator or a search engine: instead of an API for looking up a fact, the tools are things like "open this file," "apply this change," or "run this command and report what happened." This is precisely what compensates for the limitations of a bare model call — it has no way to read a file it wasn't shown or to check whether code it wrote actually runs, but an agent with file-reading and code-execution tools can do both, by making additional, purpose-specific calls to the model in between.

Memory and persistent context. An LLM, left on its own, does not remember anything between separate calls unless that information is explicitly resupplied. A coding agent addresses this not by giving the model memory it does not have, but by having the surrounding system track relevant state — which files have been read or changed, what the original task was, what has already been tried — and re-inserting the necessary parts of that state into each new call. From the developer's point of view, the agent appears to "remember" earlier parts of a multi-step task; underneath, this is the agent's surrounding system doing the remembering on the model's behalf, one call at a time.

Coding agents specifically. Because software engineering tasks are inherently complex, modular, and often require coordinating multiple steps, they were among the first domains where this kind of agent architecture was applied in practice. A coding agent built this way can be handed a task description substantially larger than a single function — implement a feature, fix a reported bug, refactor a module — and is expected to work through it the way the loop above illustrates: inspecting the relevant parts of a codebase, making a change, running the project's own tests or build tools to check the result, and revising the change if that check fails, without a human needing to manually restate context or re-supply files at every step.

Single-agent versus multi-agent workflows. So far, this section has described an agent as a single loop: one model, cycling through reasoning, action, and observation to work through a task. Not every coding agent is structured this way. Some development environments instead coordinate several such loops at once, each running with a different role, a different set of tools, or a different part of the task — for instance, one agent responsible for planning an implementation, and a separate one responsible for carrying it out. In this arrangement, the agents do not merely take turns; each is a full reasoning-action-observation loop in its own right, and the environment is responsible for passing relevant output from one agent to another rather than leaving a single model to manage every concern at once.

2.3 Multi-Agent Systems and Role Specialization

The previous section closed on a distinction: a coding agent can operate as a single loop working through an entire task alone, or several such loops can be coordinated together, each with a distinct role. This section unpacks that second case.

Why split a task across several agents. A single agent handling an entire software task end to end — understanding the requirement, planning an approach, writing the implementation, and judging whether the result is any good — has to carry every one of these concerns within the same loop and, ultimately, the same context window. A conversation focused on breaking down what a feature should do is a different kind of reasoning from a conversation focused on writing the actual code for it, and both are different again from critically checking whether that code is correct. Asking one agent to move fluidly between all of these within a single continuous context means each concern competes for the same limited space and the same undifferentiated set of instructions. Splitting the task across multiple agents, each responsible for one of these concerns, allows each agent's context, instructions, and available tools to be tailored narrowly to what that concern actually requires, rather than one generalist agent attempting all of them at once.

Role specialization as an organizing principle. In systems built this way, agents are not interchangeable copies of one another carrying out the same job in parallel; each is assigned a distinct role with its own responsibility. A pattern that recurs across the literature on this kind of system is a division into roughly three roles: a *planner*, responsible for breaking a high-level task down into smaller, well-defined subtasks; one or more *executors*, responsible for actually carrying out a given subtask, such as writing a specific piece of code; and a *critic* or *reviewer*, responsible for checking a completed subtask before it is accepted. A representative example of this approach applied specifically to software development is MetaGPT, a framework that assigns agents distinct roles modeled on a real software team — such as product manager, architect, engineer, and QA — and has them proceed through the same sequence a human team would, rather than leaving the agents to work out a division of labor on their own; this structure allows agents with more specialized responsibilities to verify intermediate results and catch errors along the way.

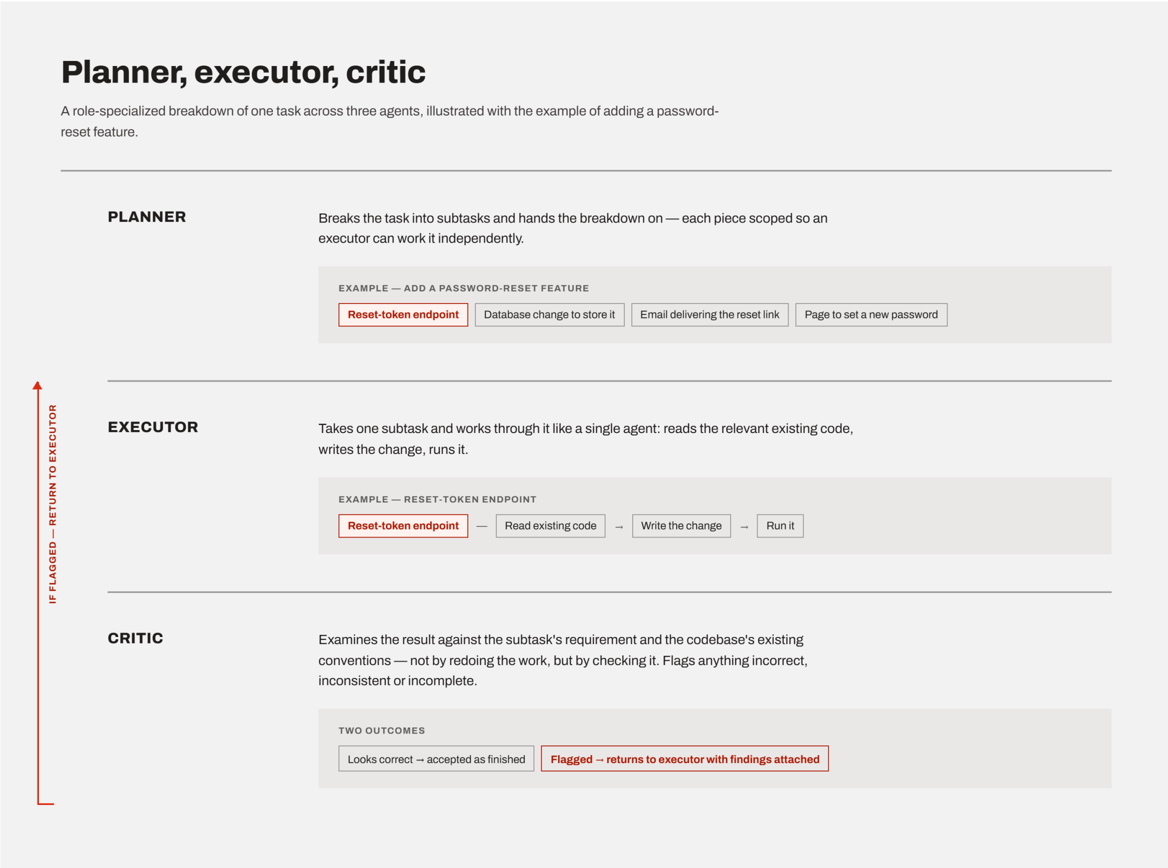

Figure 2.1: Role specialization across a planner, executor, and critic agent, illustrated with the example of adding a password-reset feature.

A concrete illustration. Consider a task such as "add a password-reset feature to the application." In a role-specialized setup, a planner agent might first break this down into subtasks — for instance, an endpoint that generates a reset token, a database change to store it, an email that delivers a reset link, and a page that lets the user set a new password — and pass that breakdown on. An executor agent then takes one such subtask, for example the reset-token endpoint, and works through it much like the single-agent loop described in the previous section: reading relevant existing code, writing the change, and running it. Once that subtask is implemented, a critic agent examines the result — not by redoing the implementation, but by checking it against the subtask's requirement and against the codebase's existing conventions, flagging anything that looks incorrect, inconsistent, or incomplete. If the critic finds a problem, that subtask goes back to the executor with the critic's findings attached, rather than being accepted as finished.

Coordination through structured hand-off. It is worth being precise about what "multiple agents working together" actually means in this architecture, since it is easy to imagine something closer to several people talking in real time. In practice, coordination usually happens through structured hand-off of artifacts rather than open-ended conversation: the planner's output is a concrete plan, passed as input to an executor; the executor's output is a concrete piece of code, passed as input to a critic; the critic's output is a concrete piece of feedback, passed back as input to the executor if changes are needed. Each agent is still, on its own, a self-contained reasoning-action-observation loop of the kind described in the previous section — what changes is that the input to one agent's loop is deliberately constructed from another agent's output, rather than from the original human instruction alone.

Trade-offs. Coordinating several agents this way is not without cost. Because each agent's work becomes an input to the next agent's reasoning, an error introduced early in the pipeline — an incomplete plan, a misunderstood requirement — can silently propagate through every later stage rather than being caught immediately, particularly if a later agent has no way of seeing the original task and can only work from what the previous agent handed it. A recent empirical study of exactly this kind of planner-executor-critic pipeline found that when a solution passes sequentially through such a pipeline, errors introduced at one stage can quietly carry through to the next, motivating close attention to which agent is actually responsible when a failure occurs. Beyond error propagation, splitting a task across several agents also means more individual LLM calls are needed to complete the same piece of work, and the system as a whole depends on each agent correctly understanding what the previous agent handed it — a coordination requirement that a single agent working alone does not face.

Taken together, role-specialized multi-agent systems trade the simplicity of a single continuous loop for narrower, more focused reasoning at each stage, at the cost of additional coordination overhead and new ways for errors to enter and spread through the process.

2.4 Human-in-the-Loop and Human Oversight in AI-Assisted Development

As we already saw, splitting a task across several agents does not remove the risk of an early mistake propagating undetected through the rest of a pipeline — it only changes where in the process an error is likely to be introduced. This raises a question that applies just as much to a single agent working alone as it does to a coordinated multi-agent pipeline: at what point, and in what way, does a human actually enter this process?

Human-in-the-loop as a design choice, not an incidental fact. It is tempting to describe human oversight loosely, as "a person eventually looked at the result." But in the literature on human-AI collaboration, human-in-the-loop (HITL) refers to something more specific: a deliberate design decision about where, in an otherwise automated process, human judgment is required before the process is allowed to continue, and what form that judgment takes. Human-in-the-loop AI combines automated capability with human oversight, feedback, and decision-making at various stages of a system's operation, rather than treating human involvement as something bolted on only at the very end. This means the relevant question for any AI-assisted workflow is not simply "is a human involved?" but "at which stage, and with what authority?"

Forms of oversight. Human involvement can take noticeably different forms, and these are worth distinguishing rather than lumping together as "review." At one end, an agent's output can be accepted automatically, with no human check at all. At the other end, a process can require explicit human approval before it is allowed to proceed to the next stage — the process pauses at a defined point and genuinely waits, rather than continuing regardless. In between sits review after the fact: a human examines a finished output once it already exists, and can accept, correct, or discard it, but the work leading up to that point was not gated on their approval. The literature draws a related distinction between keeping a human directly in a decision cycle versus having a human monitor an otherwise autonomous process and step in only when something looks wrong. Applied to the multi-agent example already introduced — a planner breaking a task into subtasks, an executor implementing one, a critic checking it — a human could sit at any of several points: approving the plan before any code is written, reviewing each subtask only after the critic has already accepted it, or not reviewing anything until the entire feature is finished.

Why placement matters, not just presence. The distinction between these forms is not merely procedural. If a human's only review happens once every subtask is finished, an error introduced at the very first step — a plan built on a misunderstood requirement — has already shaped every subtask built on top of it by the time anyone catches it, and correcting it may mean discarding work that depended on the flawed plan. Reviewing at an earlier gate, such as approving the plan itself before any implementation begins, catches the same class of error while it is still cheap to correct. This mirrors a long-standing observation in traditional software engineering, where a mistake caught during requirements or design is generally far less costly to fix than the same mistake caught only after it has been implemented and built upon. The same logic carries over directly to AI-assisted development: where a human is placed in an agent's workflow determines not just whether an error can be caught, but how much of the surrounding work has already been built on top of it by the time it is.

What human judgment contributes that the agent cannot supply on its own. As already discussed, an agent has no way of verifying its output against anything beyond what was explicitly given to it — it can check that its code runs, or that it satisfies a stated instruction, but it has no independent way of judging whether that instruction actually captured what was intended. A human reviewer can catch a different class of problem: cases where an agent followed its instructions correctly, and its output is internally consistent and free of obvious errors, but still does not achieve what was actually meant, because the instruction itself was incomplete, ambiguous, or simply did not anticipate a particular situation. This is a failure mode no amount of agent-side verification can catch on its own, since the agent has nothing else to check its work against.

Taken together, human oversight in AI-assisted development is best understood as a variable that can be tuned — in its placement, its frequency, and the amount of authority it carries at each point — rather than as a single, fixed ingredient that is either present or absent.

2.5 Prompt Engineering

As we already saw, oversight determines when a human checks an agent's work. This section addresses something further upstream: what the agent is actually told to do in the first place, before any output exists to check at all.

Prompt engineering as a deliberate practice. An LLM's behavior is shaped entirely by what is given to it as input — there is no other channel through which a person can influence what it produces, short of retraining the model itself. Prompt engineering is the practice of deliberately designing that input — the instructions, framing, and surrounding context supplied to the model — to reliably steer its output toward what is actually wanted. This involves crafting task-specific instructions, known as prompts, to guide a model's output without altering its underlying parameters, allowing the same pre-trained model to be adapted to very different tasks purely through how it is asked. It is worth being explicit that this is not the same as simply phrasing a request politely or clearly by ordinary conversational standards — a request can read as perfectly clear to a human and still leave room for an agent to settle on an interpretation the human did not intend.

System prompts versus task-specific instructions. In practice, an agent's input is typically built from two distinct layers. A system prompt sets persistent context that applies across an entire session — the agent's overall role, general rules it should follow, or background information about the project it is working in. A task-specific instruction is the individual request made within that session — "add input validation to the registration form," for instance. The two serve different purposes: a system prompt shapes how the agent behaves in general, while a task instruction specifies what it should do right now. A poorly designed system prompt can undermine an otherwise precise task instruction, since the agent is still reasoning within whatever general framing the system prompt established — and, conversely, even a well-designed system prompt cannot compensate for a task instruction that leaves the actual requirement genuinely unclear.

Prompt precision and ambiguity. An underspecified instruction does not cause an agent to visibly fail or ask for clarification in the way one might expect; more often, it causes the agent to quietly settle on one interpretation among several equally plausible ones, without flagging that a choice was made at all. Consider a task instruction such as "write a function that returns the maximum value from a list." This reads as unambiguous at first glance, but it leaves a concrete question open: what should happen if the list contains a mix of types — integers, floating-point numbers, and strings together? Should the function return the largest number regardless of the other elements, or the largest value once every element is compared as if it were text? A precise instruction removes this gap by stating the assumption explicitly — for instance, "assume the list contains only numeric values" — while an imprecise one leaves the agent to decide silently. This is not a hypothetical concern: recent empirical work on LLM-based code generation found that models frequently do not respond to this kind of ambiguity by producing varied, inconsistent attempts — which would at least signal that something in the task was unclear — but instead consistently commit to a single interpretation, producing code that is internally coherent and looks correct, while still not matching what was actually intended. This failure mode, in which a model converges on one confident but potentially wrong reading of an ambiguous task rather than exposing the ambiguity, has been documented across multiple established code-generation benchmarks. The practical consequence is that an underspecified prompt is unlikely to produce an agent that visibly struggles — it is more likely to produce an agent that appears entirely confident while quietly having answered a different question than the one that was meant.

Context design. Context is a separate concern from the system prompt versus task instruction distinction above — it can be placed in either layer, and is about what needs to be said, not where it is said. Precision addresses whether an instruction states the actual requirement clearly; context design addresses something adjacent to it — whether the agent has been given the surrounding information it needs to act on that instruction appropriately, even when the instruction itself is perfectly precise. An instruction such as "add a new API endpoint for retrieving a user's order history" can be entirely unambiguous about what the endpoint should do, while still leaving the agent without the information it would need to implement it consistently with the rest of the project — the existing naming conventions for endpoints, the authentication pattern already used elsewhere, the framework and libraries already in use, or the format other endpoints use for error responses. Given only the bare instruction, an agent has no way to know these things and will invent plausible-sounding defaults of its own; given the same instruction alongside a short excerpt of an existing, similar endpoint, the agent has a concrete pattern to follow instead of a gap to fill. Precision and context therefore address two different gaps: precision narrows what is being asked for, while context supplies what the agent would otherwise have to guess.

Taken together, prompt engineering determines the starting conditions an agent begins a task from — not whether its output will later be checked, but how much ambiguity and missing context that output is generated under in the first place.

2.6 Software Architecture in the Context of AI-Generated Code

Prompt engineering and context design, as discussed above, address what an agent is given before it acts on a single task. This section addresses a different kind of problem — one that does not show up within any single task, but only becomes visible once many separate tasks have accumulated over time.

Architectural conformance and drift. A codebase's architecture is not just its current structure, but the set of intended structural decisions a project is meant to follow — how components are organized, how they are meant to communicate with one another, what belongs where. Architectural conformance is the degree to which the actual, implemented code continues to follow these intended decisions. The opposite of conformance is not a single event but a gradual process: the literature distinguishes architecture erosion, caused by decisions that directly violate the architecture, from architecture drift, caused by decisions made without awareness that a governing architecture exists at all — the outcome in both cases is the same, a growing gap between how the system was meant to be structured and how it actually is.

Why this is a cumulative problem, not a single-instance one. A single missing piece of context, of the kind discussed earlier, causes one decision to be made without the information it needed. Architectural drift is a different scale of problem: it is what results when many such individually reasonable decisions, made across many separate tasks over time, each fail to align with a structural pattern established somewhere else. No single one of these decisions needs to be wrong on its own terms — each may correctly satisfy the specific instruction it was given — for the codebase as a whole to gradually lose its coherence.

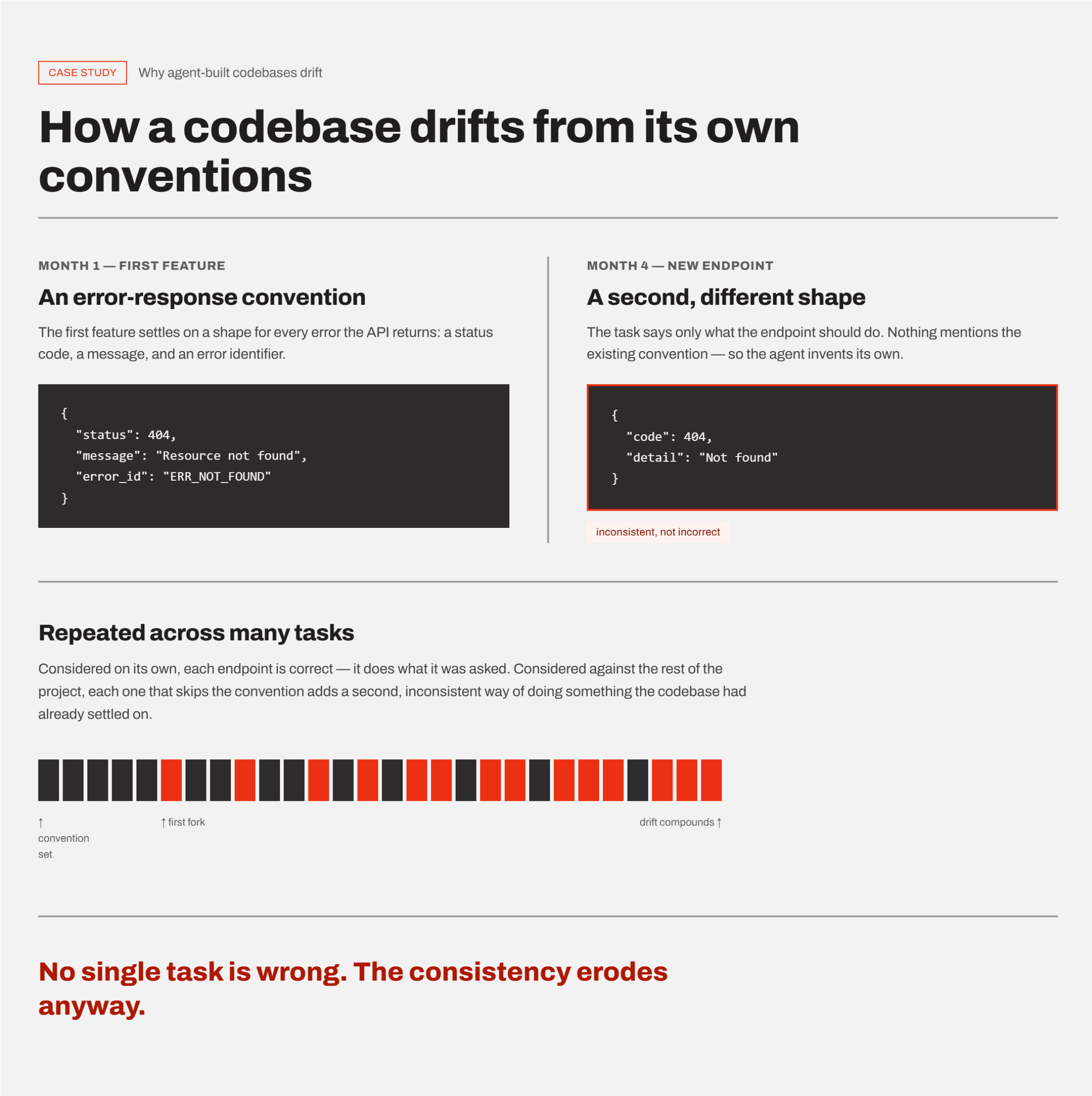

Figure 2.2: How individually reasonable decisions, made across many separate tasks without shared context, accumulate into architectural drift.

A concrete illustration. Suppose a project's first feature introduces a particular way of formatting error responses from its API — a consistent shape including a status code, a message, and an error identifier. Months later, a different task asks an agent to add a new endpoint elsewhere in the project. If that task's instruction says only what the endpoint should do, and nothing about the project's existing error-handling convention, the agent has no way of knowing that convention exists, and may reasonably invent its own, differently structured error response. Considered on its own, this new endpoint may be entirely correct — it does what it was asked to do. Considered against the rest of the project, it has quietly introduced a second, inconsistent way of doing something the codebase had already settled on. Repeated across many features and many separate tasks, this is how a codebase drifts away from its own established patterns without any single change being clearly at fault.

Why this is not unique to AI agents, but is not the same either. Architectural drift is not a problem invented by AI-assisted development — human developers drift from an established architecture over time as well, particularly on projects with many contributors or a long history. What differs is how familiarity with the architecture is maintained. A human developer working on a project over weeks or months accumulates an implicit sense of its existing patterns simply by being continuously immersed in it. An agent's grasp of those same patterns, by contrast, is only as good as what is deliberately reconstructed for it at the start of each task — LLMs applied to code generation frequently produce code in isolation, without an understanding of project-specific architectural patterns, and this contextual disconnect is a documented source of style and structural inconsistency across generated code. Without that reconstruction, an agent does not retain the accumulated sense of "how things are done here" that a long-tenured human contributor builds up passively.

Architectural drift is, by its nature, not something a single well-crafted prompt can solve. Precision and context, as discussed earlier, improve the conditions under which one task is carried out, but drift is a property that emerges across many tasks over time — preventing it requires some form of constraint or convention that persists across all of them, rather than being restated freshly, or not restated at all, with every new instruction.

2.7 Software Testing and Test Validity

Architectural drift, discussed above, concerns whether generated code stays structurally consistent with the rest of a project over time. This section addresses a related but distinct question, one that applies even to a single, well-isolated piece of code: when a test passes, how much does that actually tell us?

A passing test versus a valid test. A test passing means only one specific thing: the code, when run against that particular test, produced the outcome the test checked for. It says nothing about whether the test checked for the right thing in the first place. A test suite can run to completion, report every test as passing, and even claim high coverage of the codebase, while still failing to verify that the software does what it was actually supposed to do — because the tests themselves never asked the right question. This gap, between a test passing and a test validating the intended behavior, is the central concern of this section.

Why this gap is particularly relevant when tests are agent-generated. As established earlier, an LLM has no inherent mechanism for verifying its own output against anything beyond what it was given — it can produce plausible, syntactically correct output, but has no independent way of confirming that output is actually right. A generated test is not exempt from this limitation; it is itself just another piece of generated code, subject to the same constraint. An agent asked to "write tests for this feature" can produce a test suite that runs successfully and reports strong coverage numbers, while the tests themselves check only shallow properties of the code rather than its intended behavior. This is not a hypothetical risk: the literature has found that code coverage is weakly correlated with the effectiveness of LLM-generated tests in detecting bugs, meaning a test suite can exercise a large portion of the code without being any good at catching cases where that code is wrong.

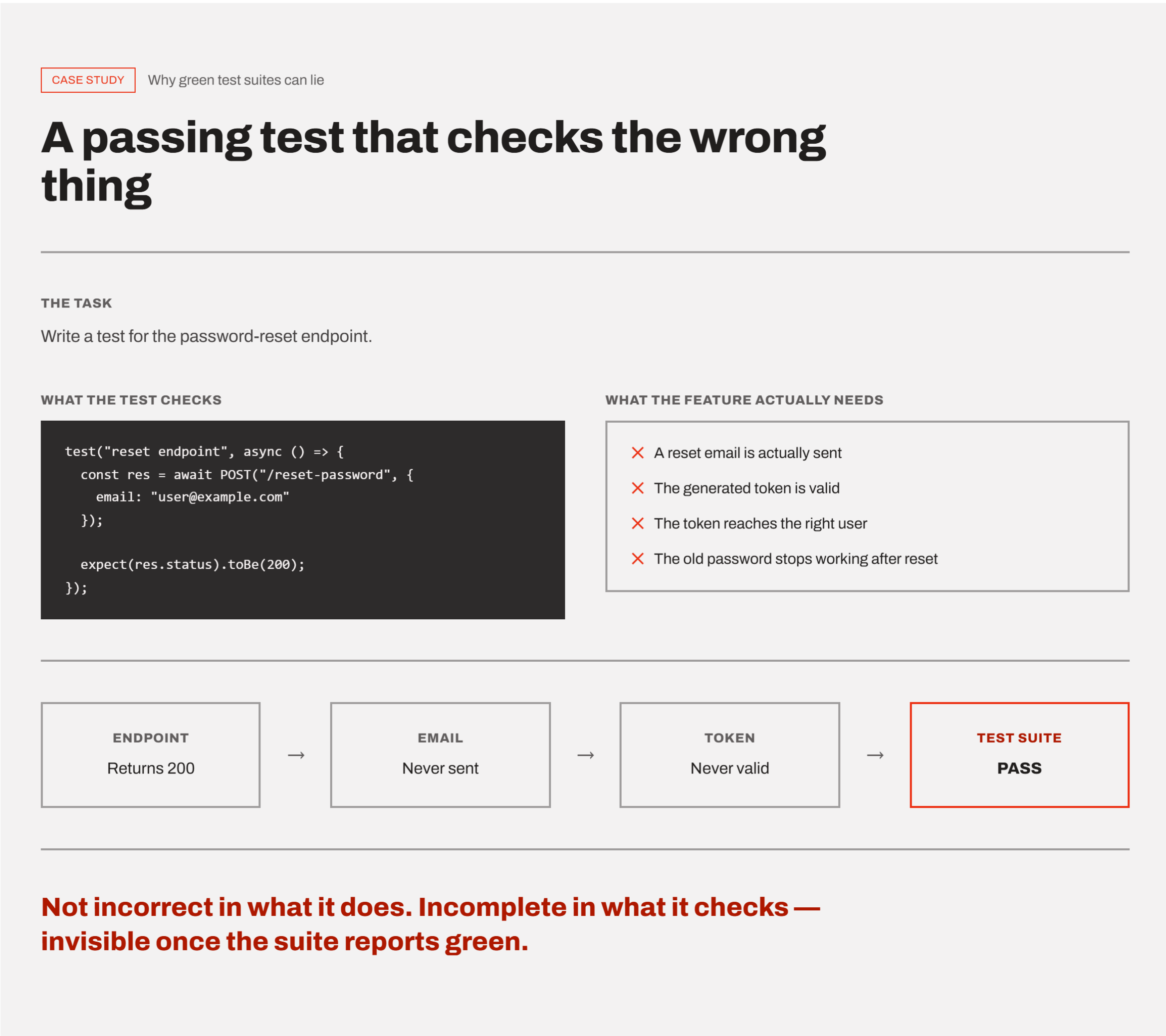

Figure 2.3: A passing test suite versus a test suite that actually validates the intended behavior.

A concrete illustration. Consider an agent asked to write tests for a password-reset endpoint. A test that only checks that calling the endpoint returns a successful HTTP status code will pass regardless of whether a reset email was actually sent, whether the generated token is valid, or whether the token even reaches the right user — the endpoint could be broken in exactly the way that matters to the feature's purpose, and this test would still report success. The test is not incorrect in what it does; it is incomplete in what it checks, and that incompleteness is invisible from the outside once the test suite reports green.

A related but distinct failure mode: tests that validate the implementation rather than the requirement. A separate risk arises when the same agent, or the same generation process, produces both the implementation and its tests. In this situation, a test can end up confirming that the code behaves the way the code happens to behave, rather than the way the original requirement intended. Empirical work on LLM-based test generation has documented this concretely: when a model is prompted with a buggy implementation, it can treat the faulty behavior as intended and generate a test that asserts the incorrect outcome, rather than the behavior the function was actually supposed to have — the resulting test still passes, and still contributes to coverage, while actively encoding the wrong behavior as if it were correct. This is a different problem from incomplete coverage: the test is not merely checking too little, it is checking the implementation against itself rather than against the original, independent requirement it was meant to satisfy.

Taken together, these observations point to the same underlying principle: whether a test suite is any good cannot be judged from whether it passes, or even from how much of the code it exercises, but only by checking it against the original requirement the software was meant to fulfill — a check the test suite itself, by construction, cannot perform on its own behalf.

2.8 OpenAPI Specifications and Documentation Drift

Test validity, discussed above, concerns whether a piece of code's behavior matches what was actually intended. This section addresses a related concern one level removed from the code itself: whether that behavior stays accurately described, in an artifact other people and other systems rely on, once the code has changed.

What an OpenAPI specification is. An OpenAPI specification is a structured, machine-readable document that describes an HTTP API's capabilities — its endpoints, the inputs each one expects, and the responses it returns — allowing both humans and computer programs to understand what a service offers without needing to read its source code or inspect its network traffic directly. Because the specification is machine-readable rather than free-form prose, it is not only read by developers as documentation; it is also consumed directly by tooling — generating client code that calls the API automatically, generating interactive documentation pages, or validating that incoming requests match what the API claims to accept.

Why a specification and its implementation can diverge at all. An OpenAPI specification is, in practice, a separate artifact from the code that implements the API it describes — typically a separate file, written and updated by hand or by a separate generation step, rather than something the programming language itself enforces consistency with. Nothing structurally prevents the two from disagreeing: a specification file has no built-in awareness that the code it describes has changed, and the code has no built-in awareness that a specification file describing it exists at all, unless something is explicitly built to connect the two.

Documentation drift. This divergence — between what a specification claims about a system and what the system actually does — is generally referred to as documentation drift, and its underlying cause is structural rather than accidental: code changes are continuously exercised by compilers, automated tests, and continuous integration pipelines, so problems introduced into the code tend to be caught quickly, while documentation changes are not subject to any of these automatic checks and are instead updated manually, opportunistically, and often incompletely. This is conceptually the same shape of problem as the architectural drift discussed earlier — a gradual gap opening between an intended, documented state and the actual one — except here the divergence is not within the code itself, but between the code and a separate document describing it.

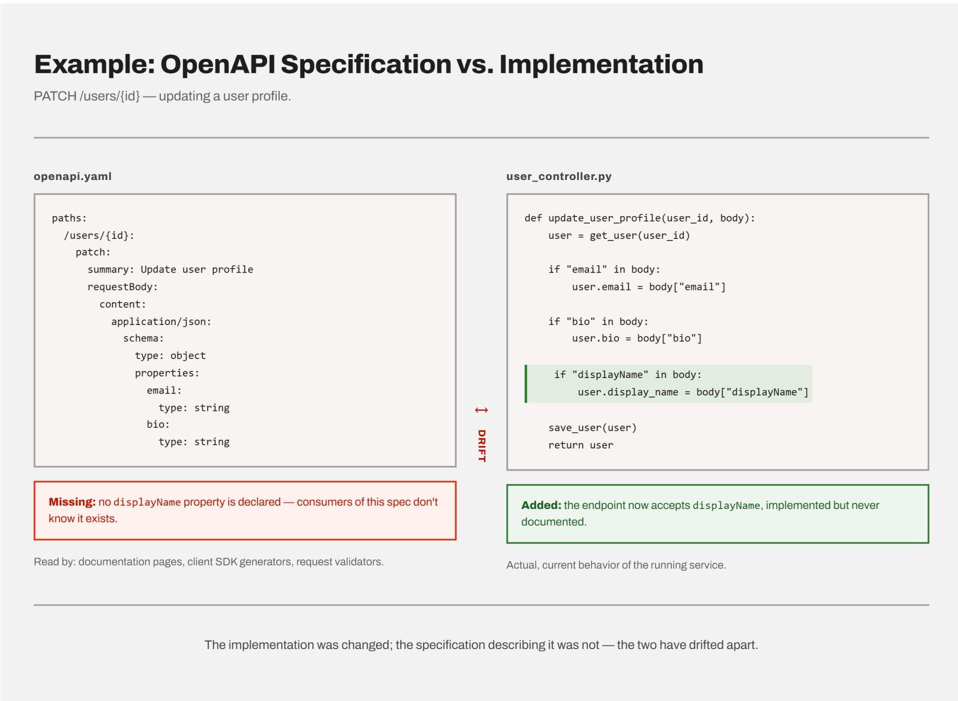

Figure 2.4: An implementation change accepted by the code but never reflected in the OpenAPI specification describing it.

A concrete illustration. Suppose an agent is instructed to add a new optional field to an existing endpoint's request body — for instance, allowing a user profile update to optionally include a display name. Implementing this change means modifying the code that handles the request. Updating the OpenAPI specification to reflect that this new field now exists is a separate, additional step, and nothing about the instruction to "add the field" inherently requires the agent to also touch the specification file, unless it is explicitly told to or has been given a reason to check. If it is not, the implementation now accepts a field the specification does not mention — a consumer relying on the specification, whether a human developer or an automatically generated client, has no way of knowing the field exists at all, or may be shown an inaccurate picture of what the endpoint actually expects or requires.

Why this matters beyond documentation itself. With OpenAPI specifications specifically, a stale specification is not only a misleading document for a human reader — because these specifications frequently drive automated tooling directly, a specification that no longer matches the implementation can cause that tooling to actively misbehave: an auto-generated client built from the outdated specification may fail to send the new field at all, or a request-validation step may reject genuinely valid requests because the specification it is checking against has not been updated to allow them.

Like architectural drift, documentation drift is not something a single correctly specified instruction can prevent on its own — updating an implementation and updating its specification are two separate actions, and nothing automatically ties the two together across the many separate tasks that touch an API over the course of a project, unless something is deliberately built to keep them linked.

3 Methodology

3.1 Structure of the Practical Project (Two Sub-Projects)

This thesis is interested in how the structure of human-agent collaboration affects the quality of the resulting software — not in how capable one particular model is at isolated coding tasks. Standard coding benchmarks such as HumanEval, mentioned earlier in the foundations, or SWE-bench score an agent on short, self-contained problems with a single correct answer; they aren't built to capture what happens across a sustained, multi-feature project, where architecture, testing habits, and documentation either hold up or don't over time. For that reason, this thesis instead builds a single application under two different collaboration structures — an unstructured, single-agent workflow in Part 1, and a structured, multi-agent workflow with defined review steps in Part 2 — so that any differences in the resulting software can be attributed to how the collaboration was structured, rather than to a difference in what was being built.

The constant baseline: one software idea, one application. Both sub-projects implement the same underlying application, APICraft — a fullstack API collaboration and testing workspace that allows users to construct, execute, and manage HTTP requests, organize them into collections, inspect detailed response information, and persist requests for reuse. The software idea, its core modules, and its functional and non-functional requirements were written down once, at the very start of the practical project, and given to the agent only in Part 1. This initial specification document, together with early UI mockups, is what Part 1 was built from.

Part 2 did not receive this document again. By the time it began, the codebase already existed, and a persistent set of project knowledge files — describing the application's architecture and structure — carried the same context forward without needing to restate it. So while only Part 1 saw the original specification directly, both parts trace back to that same starting point: one software idea, developed continuously across the two parts.

Part 1: unstructured, single-agent development. The first sub-project was carried out using Claude Code inside Visual Studio Code, in a conversational, single-agent workflow with minimal structural guidance beyond the software idea and requirements document itself. Part 1 covers the application's foundational scope: authentication (login, registration, sign-out), workspace and collection management, request management, the request builder (parameter, header, body, and authorization tabs), request execution, response display, and the sidebar navigation. The goal of this part was to observe how an agent behaves when given substantial implementation freedom, and where that freedom begins to produce inconsistency or architectural weaknesses — an aim that is examined in detail later.

Part 2: structured, multi-agent development. The second sub-project continues the same codebase produced in Part 1 — it is an extension, not a rebuild — but switches to Google Antigravity as an agent-first development environment and introduces an explicit collaboration structure: a three-agent workflow consisting of an Architect Agent, an Implementer Agent, and a Post-Implementation Review Agent, governed by a written set of architectural rules (an "AI Development Charter") that all three agents are required to follow. Under this structure, Part 2 both remedies architectural weaknesses identified during Part 1 and adds further scope to the application: workspace-level roles and permissions, inline request comments, an activity feed and audit logging, a performance and analytics dashboard, request export, and OpenAPI specification automation.

Artifacts produced by each part and their role in later analysis. Each part of the practical project produced its own evidentiary trail, which forms the primary source material for the comparison and evaluation that follow. Part 1 is documented through a review of its development process and the architectural problems that emerged during it, together with the reasoning applied when generated tests failed. Part 2 is documented through the structured artifacts its workflow requires by construction: an `implementation_plan.md` produced by the Architect Agent for each feature (including a mandatory design review of architecture rationale, state ownership, service ownership, testing strategy, and scalability concerns), a `walkthrough.md` and updated structure maps produced by the Implementer Agent, and a `review_report.md` produced by the Post-Implementation Review Agent against a fixed six-point checklist. Together with a persistent set of project knowledge files — an architectural rulebook and living structure maps for the backend and frontend — these artifacts make Part 2's development process directly inspectable in a way Part 1's was not.

3.2 Tools and Models Used

3.3 Evaluation Criteria

4 Part 1: Unstructured Development

4.1 Initial Situation and Objectives

4.2 Prompting Without a Structural Approach

4.3 System Prompt Context

4.4 Development of the First Feature (Login/Registration)

4.5 Test Suites and Test Case Generation

4.6 Architectural Analysis of the Generated Software

4.7 Observations: Prompt Precision, Human Oversight, Architectural Limitations

4.8 Critical Problems and the Test Failure Analysis Principle”

5 Part 2: Structured Approach (AntiGravity)

5.1 Motivation for a Structured Approach

5.2 AI Development Charter and Architectural Rules

5.3 Parallel Agent Architecture (Architect / Implementer / Post-Implementation Review Agent)

5.4 Design Reviews Before Coding (implementation_plan.md)

5.5 Testing Strategy: Force Requirement Validation & Three-Tier Testing

5.6 Project Knowledge Files

5.7 OpenAPI Specification Automation and Avoiding Documentation Drift

5.8 Identification of Architectural Problems and Weaknesses

6 Comparison of Both Approaches

6.1 Comparison: Part 1 vs. Part 2

6.2 Feature Diversity

7 Evaluation

7.1 Evaluation Criteria and Methodology

7.2 Quality of Generated Code

7.3 Quality and Validity of Generated Tests

7.4 Architectural Conformance and Documentation Consistency

7.5 Efficiency and Effort (Human vs. Agent)

7.6 Summary Assessment: Part 1 vs. Part 2

8 Best Practices for Human-Agent Collaboration

8.1 Importance of Prompt Precision and Context Design

8.2 Role of Human Oversight (Human-in-the-Loop)

8.3 Architectural Guardrails for AI Agents

8.4 Recommendations for Multi-Agent Workflows

8.5 Avoiding Documentation Drift

9 Discussion

9.1 Opportunities and Limitations of AI Agents in Software Development

9.2 Implications for Practice

10 Conclusion and Outlook

Bibliography